



Robotics & Computer Vision

6D Object Pose Estimation for Robotics

Moving beyond object detection — toward full position and orientation in 3D space.

Week 5 Learning Goals

1

Understand 6D Pose

Grasp what 6D pose means in computer vision and robotics contexts.

2

Position vs. Orientation

Distinguish between the translational and rotational components of pose.

3

Camera $\leftrightarrow$ Robot

Understand the relationship between camera pose and robot coordinate frames.

4

AI-Based Inference

Learn how deep learning models infer 6D pose directly from images.

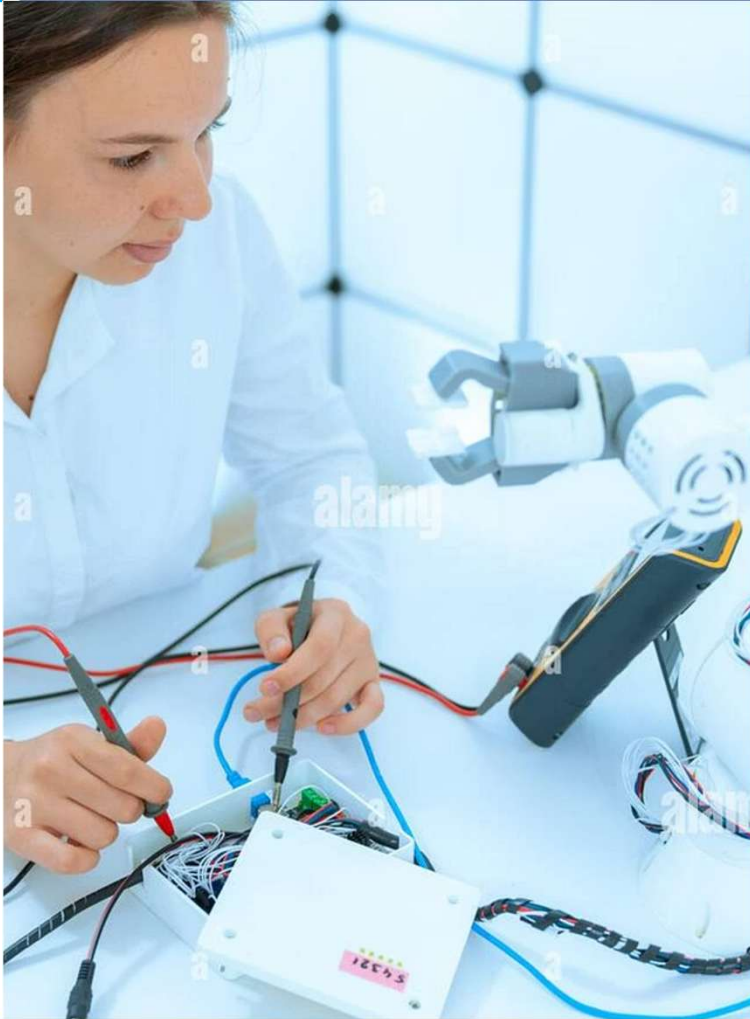
Week 5 in the Course Roadmap

Topic

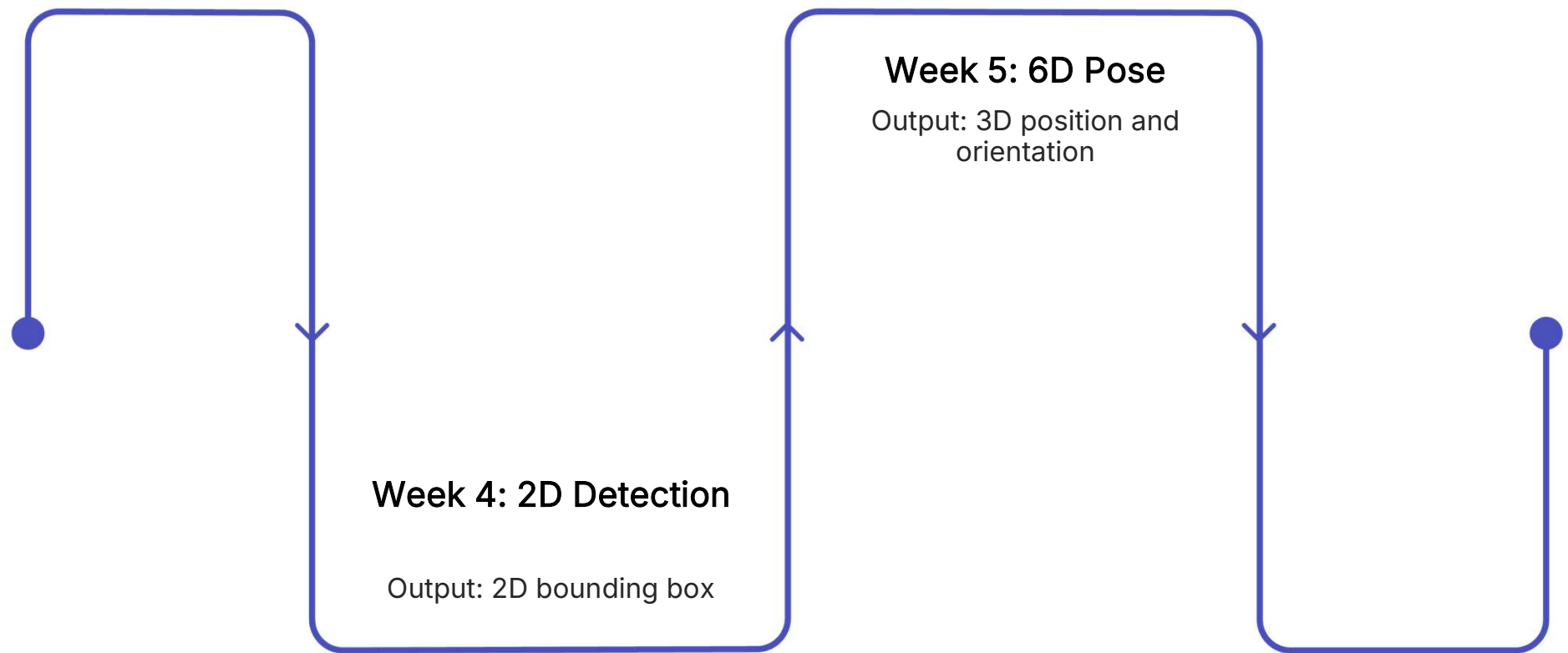
AI Vision: 6D Object Pose Estimation using Deep Learning.
The model outputs a full spatial description of each detected object.

Application Output

Position (x, y, z) and orientation (**roll, pitch, yaw**) — or a full pose matrix — relative to the camera frame, ready to serve robotic control systems.



Building on Week 4

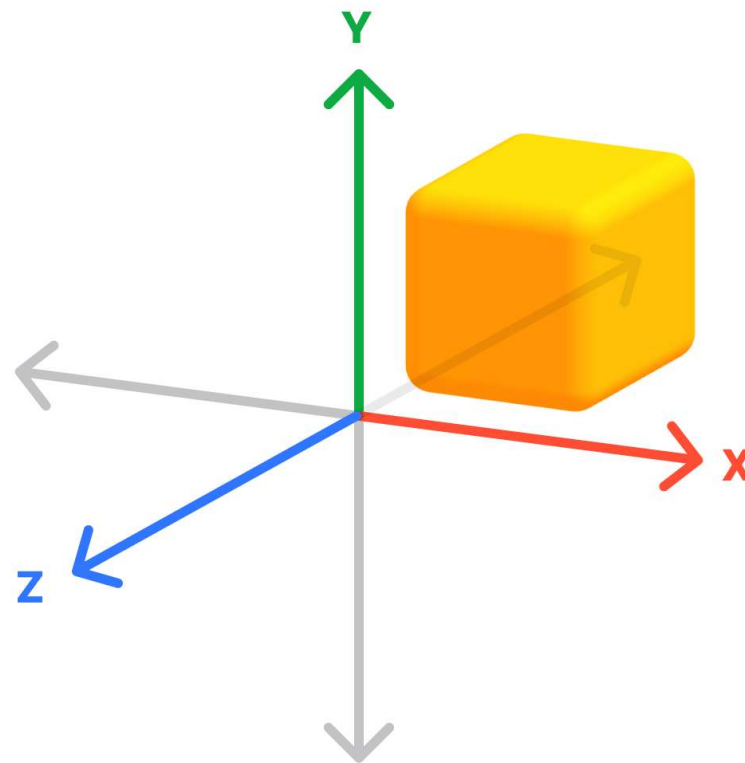


Detection tells the robot *what* the object is. Pose estimation tells it *where* the object is and *how it is oriented* — the critical step toward autonomous grasping.

PART 1

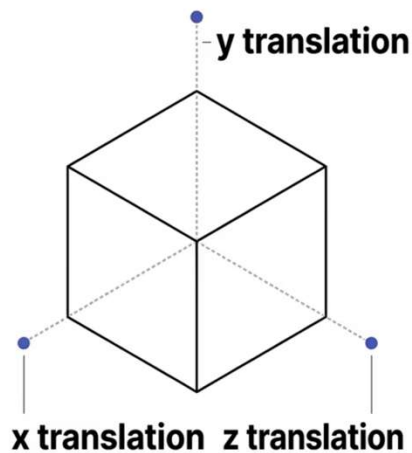
What Is Pose?

Knowing where an object is — and how it is rotated — in 3D space.

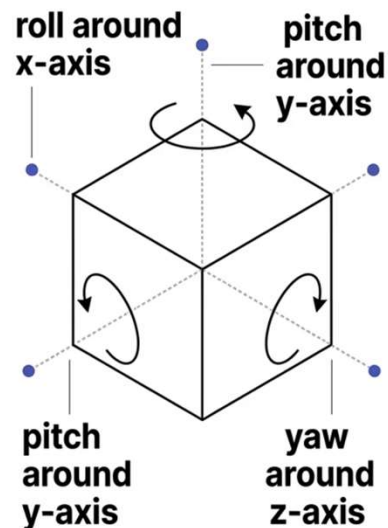


Defining 6D Pose

POSITION



ORIENTATION



All 6 degrees of freedom combined define an object's complete movement in 3D space.

Six Degrees of Freedom

A **6D pose** fully describes how an object is positioned and oriented in 3D space:

- **3 DOF for position:** x, y, z coordinates
- **3 DOF for orientation:** roll, pitch, yaw angles

Together, these six values completely characterize the spatial state of any rigid body.

Position and Orientation

Position

Specifies **where** the object is located within a reference coordinate system. Expressed as a 3D translation vector $[x, y, z]$.

Orientation

Specifies **how** the object is rotated around the axes of that space. Expressed as Euler angles, a rotation matrix, or a quaternion.

Why Robots Need 6D Pose



→ Beyond the Centroid

A 2D centroid is not enough — the robot arm must know the object's orientation to align its gripper correctly.

→ Approach Angle

Pose helps the robot calculate the correct approach vector and avoid collisions during the grasp.

→ Bin Picking & Manipulation

Full 6D pose is the essential prerequisite for reliable bin picking, assembly, and object manipulation tasks.

A rotation θ° anticlockwise around the y -axis.

$$I = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \text{ maps to } I' = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

$$J = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \text{ maps to } J' = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

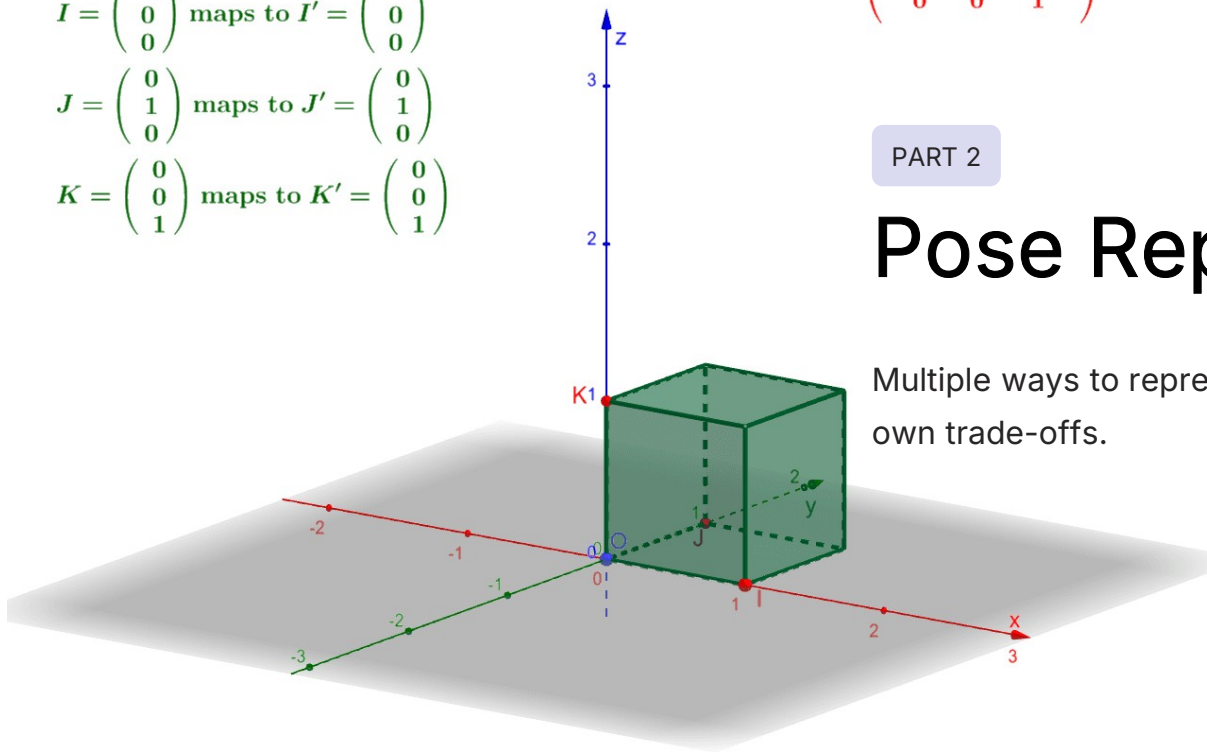
$$K = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \text{ maps to } K' = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$\text{Matrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

PART 2

Pose Representations

Multiple ways to represent the same geometric state — each with its own trade-offs.



Translation Vector

The Position Component

Position is written as the column vector $[x, y, z]^T$ — the coordinates of the object's origin within a reference frame.

The reference frame could be the **camera frame**, the **world frame**, or the **robot base frame**. Always specify which frame you are using — an ambiguous pose vector is meaningless in practice.

Camera Frame

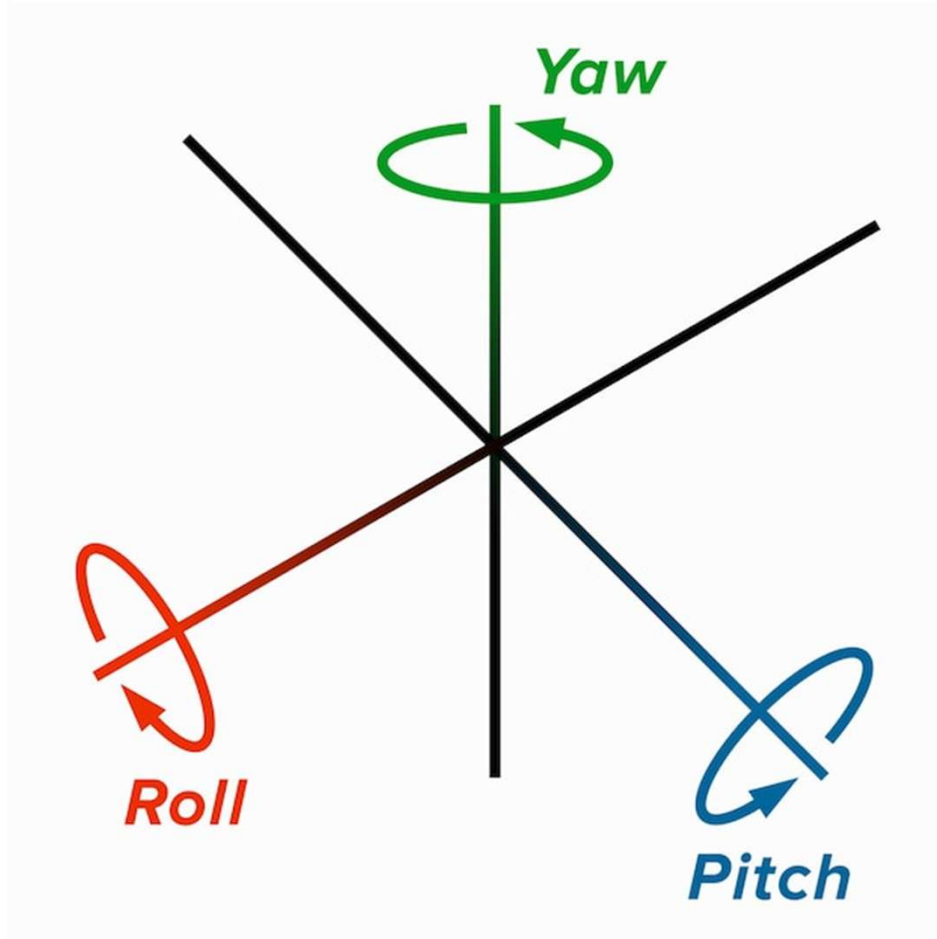
Relative to the camera lens — most common output from vision models.

World Frame

Fixed environment origin — used in simulation and mapping.

Robot Base Frame

Relative to the robot's base — required for motion planning.



Roll, Pitch, and Yaw

Euler Angles — Intuitive but Imperfect

- Roll — rotation around the x-axis
- Pitch — rotation around the y-axis
- Yaw — rotation around the z-axis

Euler angles are easy to visualize and interpret, but they can suffer from **gimbal lock** — a singularity where one degree of freedom is lost. For this reason, more robust representations are preferred in computation.

Rotation Matrix

A Robust Orientation Representation

Orientation is most reliably encoded as a **3×3 rotation matrix R** . Key properties:

- Preserves lengths and angles (orthonormal)
- Each column represents one axis of the object in the reference frame
- No singularities — very common in robotics software stacks

Column 1

Object's x-axis direction expressed in the reference frame.

Column 2

Object's y-axis direction expressed in the reference frame.

Column 3

Object's z-axis direction expressed in the reference frame.

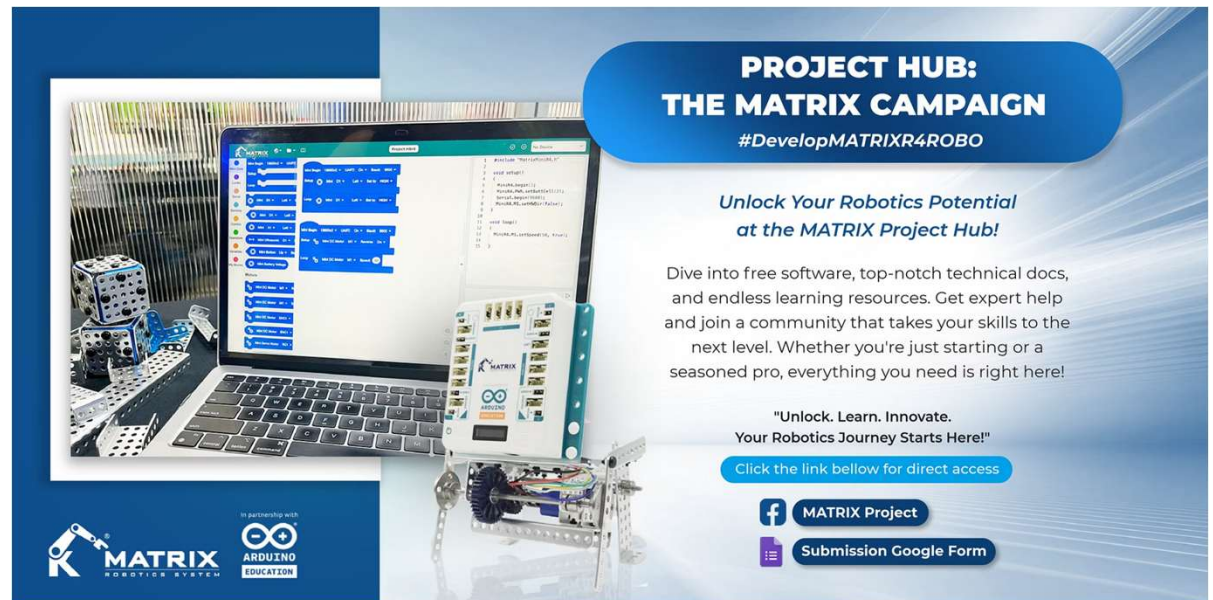
Homogeneous Transformation Matrix

The Standard 4×4 Pose Matrix

The complete pose is packed into a single **4×4 homogeneous matrix T**:

- Top-left **3×3 block** → rotation matrix **R**
- Right **3×1 column** → translation vector **t**
- Bottom row → **[0, 0, 0, 1]**

This compact representation makes it easy to **chain multiple transforms** with simple matrix multiplication — the standard in robotics and 3D graphics.



**PROJECT HUB:
THE MATRIX CAMPAIGN**
#DevelopMATRIXR4ROBO

*Unlock Your Robotics Potential
at the MATRIX Project Hub!*

Dive into free software, top-notch technical docs, and endless learning resources. Get expert help and join a community that takes your skills to the next level. Whether you're just starting or a seasoned pro, everything you need is right here!

"Unlock. Learn. Innovate.
Your Robotics Journey Starts Here!"

[Click the link below for direct access](#)

[MATRIX Project](#)

[Submission Google Form](#)

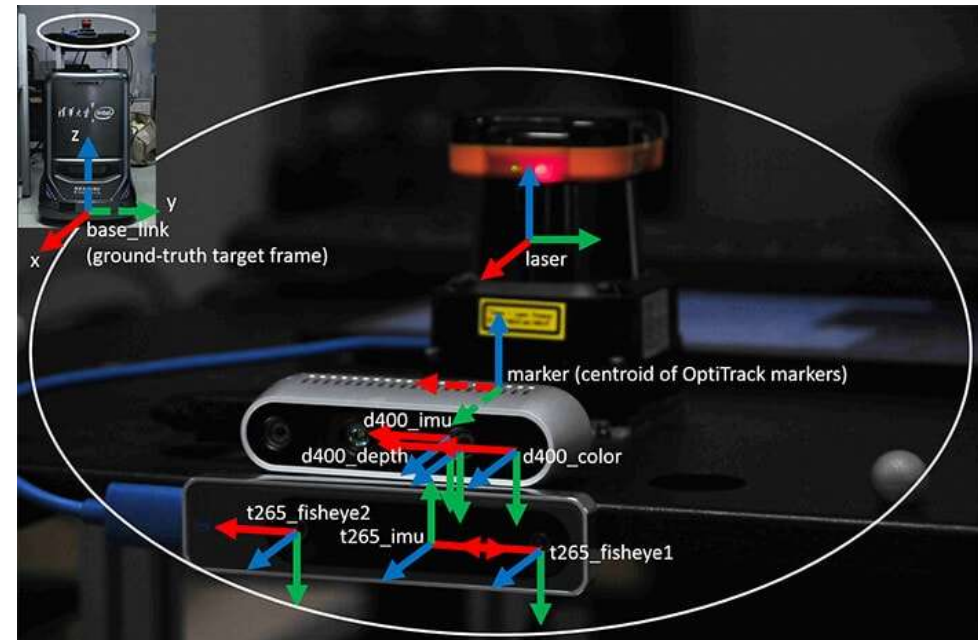
MATRIX
ROBOTICS SYSTEM

in partnership with
ARDUINO
EDUCATION

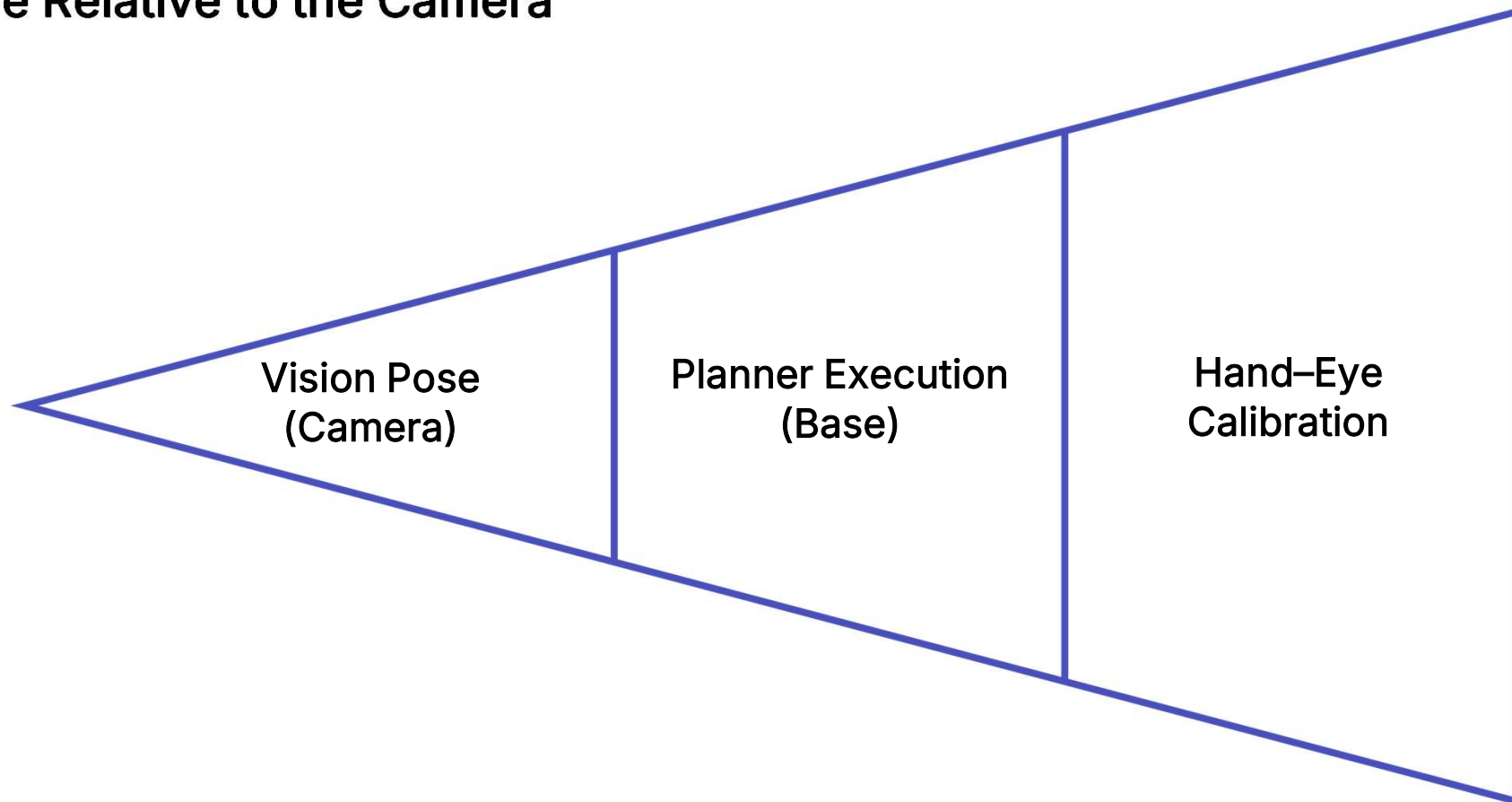
PART 3

Coordinate Frames and Cameras

Pose always belongs to a specific reference frame — context is everything.



Pose Relative to the Camera



World Frame vs. Robot Base Frame

World Frame

A **fixed global origin** for the environment. Useful for simulation, building maps, and coordinating multiple robots or sensors in a shared space.

Robot Base Frame

Attached to the **robot's base link**. This is the frame the controller needs — motion planners and inverse kinematics operate here to execute physical actions.

Why Is 2D $\rightarrow$ 3D Pose Hard?

Depth Information Is Lost

Projecting 3D onto a 2D image discards the depth dimension — this must be recovered or inferred.

Ambiguous Projections

Multiple distinct 3D poses can produce nearly identical 2D images, making the inverse problem ill-posed.

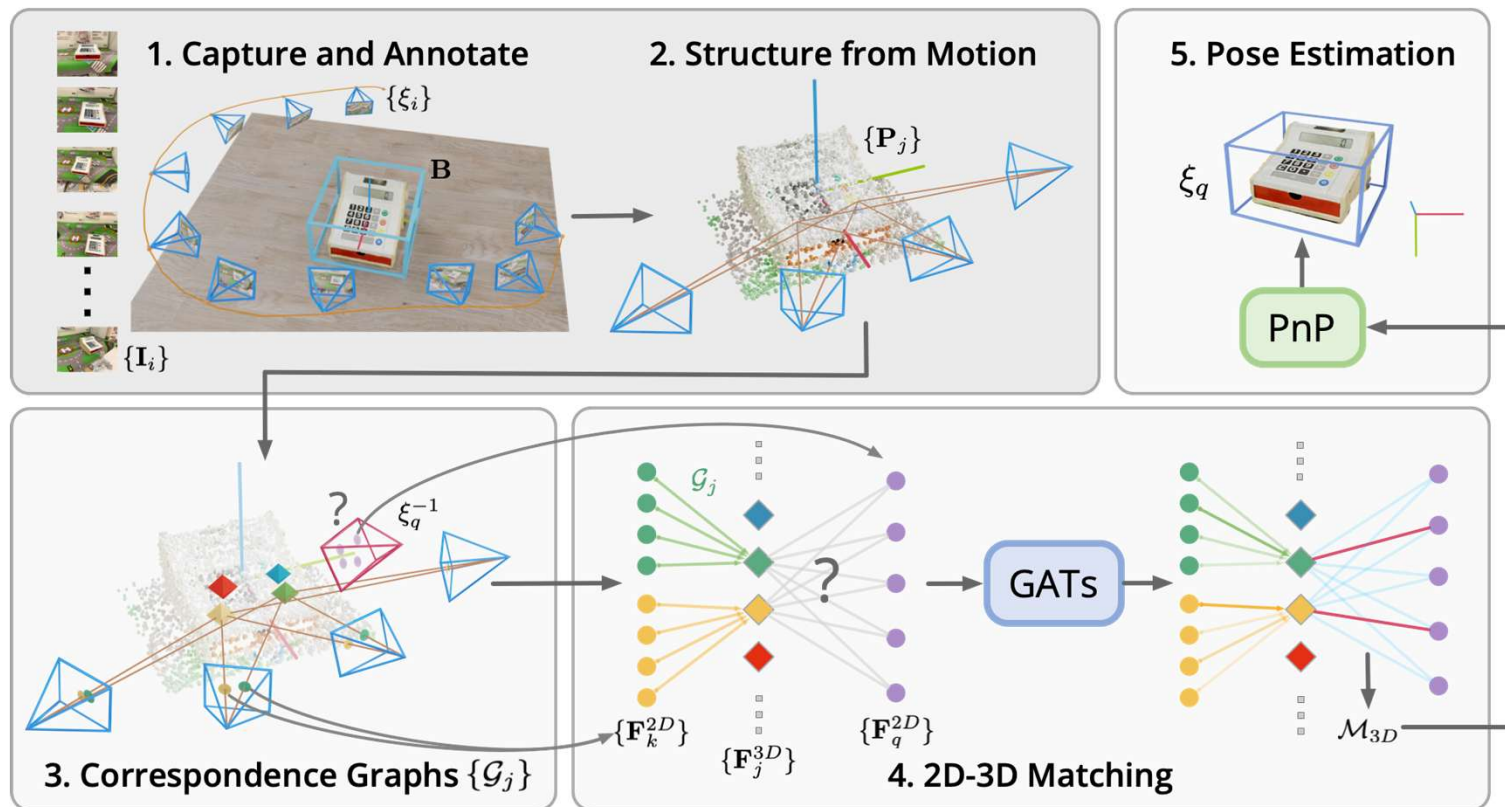
Occlusion and Reflections

Partially hidden objects and specular surfaces degrade feature quality and confuse estimators. This is why strong models or depth sensors are often required.

PART 4

Approaches to Pose Estimation

From classical geometry to end-to-end deep learning.



Classical Geometric Approach



Feature-Based Geometry

Uses known 2D–3D correspondences — for example, from a CAD model — to solve for pose analytically (e.g., via PnP).

Advantages: Interpretable, accurate when features are reliable, no GPU required.

Limitations: Requires high-quality feature detection and a trusted 3D model or known geometry.

Deep Learning Approach

End-to-End Neural Networks

A network learns to predict pose **directly from RGB or RGB-D images**, without hand-crafted feature pipelines.

Strengths: Robust to complex backgrounds and objects that are hard to model manually.

Trade-offs: Requires large labeled datasets and significant compute for training.

Direct Regression

Predict (x, y, z, r, p, y) directly from the image.

Keypoint Prediction

Detect 2D keypoints, then solve pose in post-processing.

Dense Correspondences

Map every pixel to a 3D model point for robust estimation.

RGB, Depth, and RGB-D Sensors



RGB

Cheap and widely available. No direct depth — must be inferred. Suitable for many real-world deployments.



Depth

Provides direct distance measurements per pixel. Highly informative for 3D pose but can be noisy on reflective surfaces.



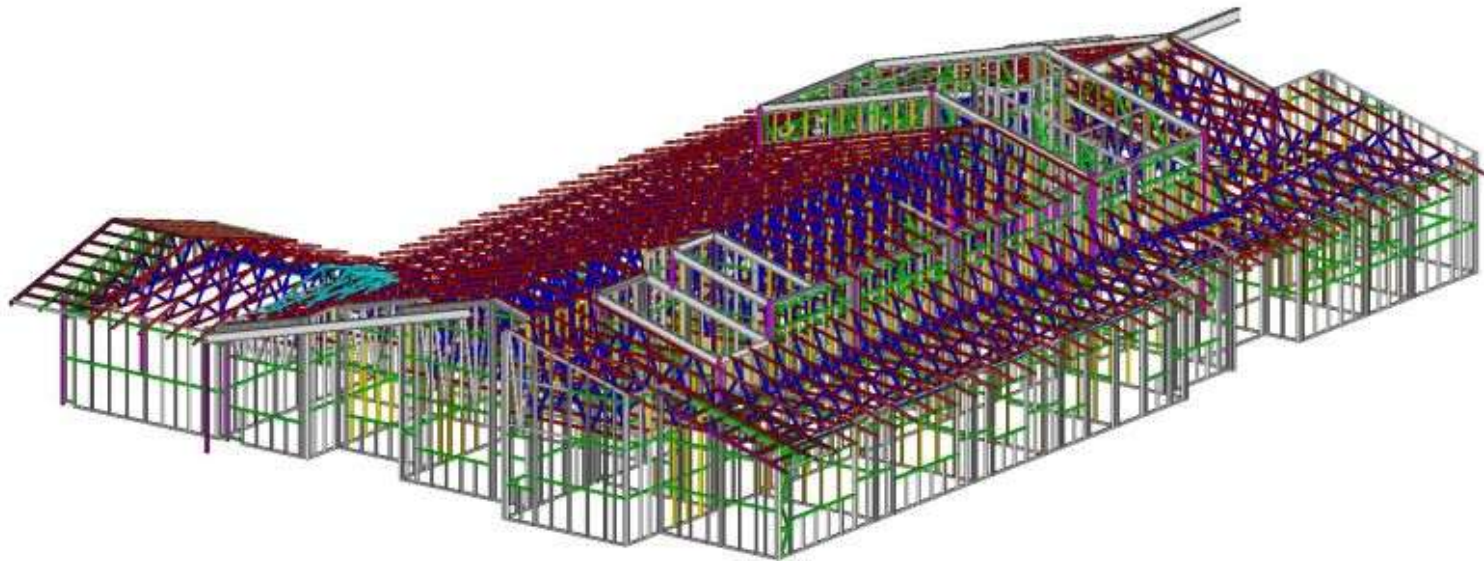
RGB-D

Combines color and geometry. Best of both worlds — supports richer feature extraction and more accurate spatial inference.

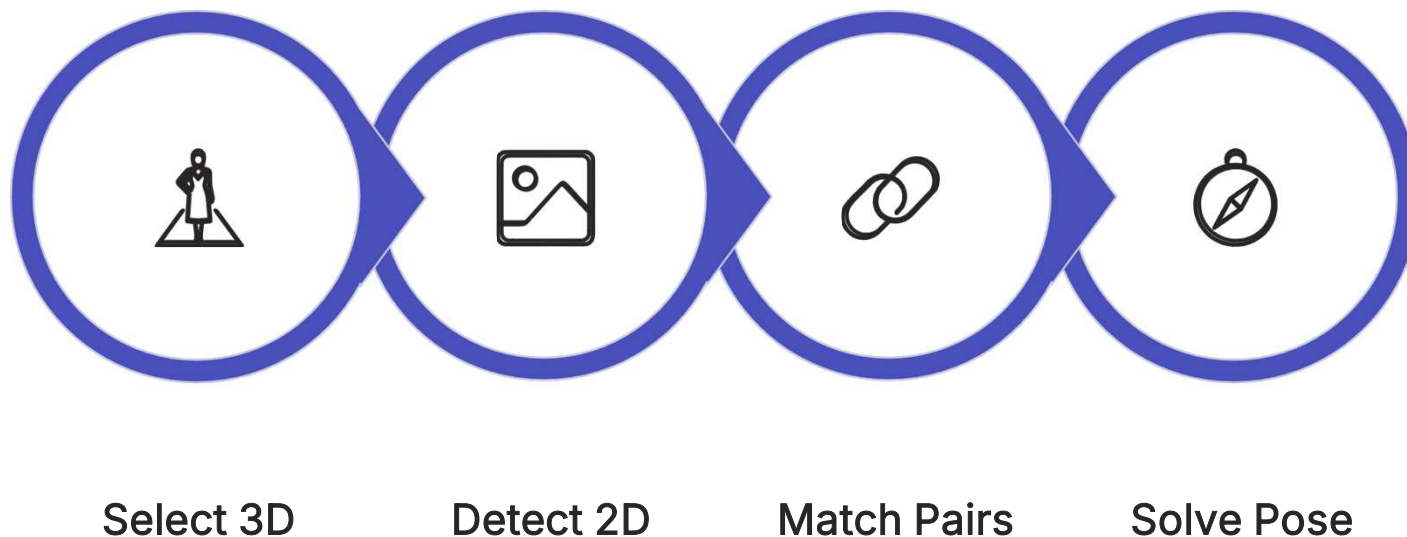
PART 5

Keypoints and PnP

A natural bridge between computer vision and robotics — geometry meets learning.



2D–3D Keypoint Correspondences



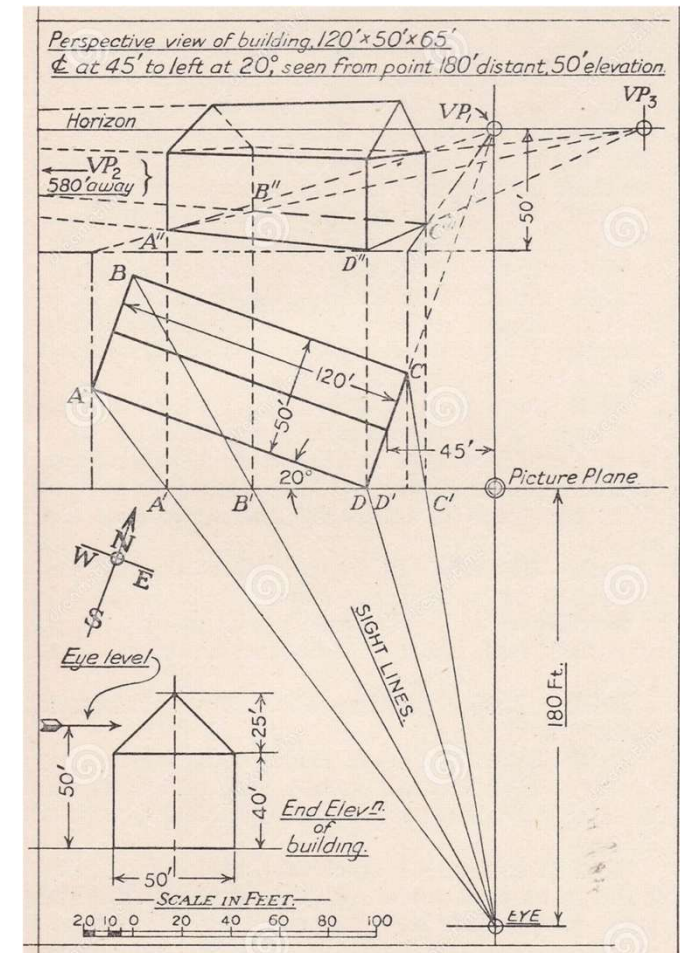
Identifying a set of well-distributed keypoints — and matching them between the 3D model and the 2D image — is the core idea behind geometry-based pose estimation. This is an excellent conceptual entry point for students.

PnP — The Intuition

Perspective-n-Point

PnP answers: given n known 3D points and their 2D projections on the image, what is the camera/object pose?

- Requires at least 4 non-coplanar point pairs
- Solved analytically or iteratively
- OpenCV provides `cv2.solvePnP()` out of the box
- Outputs rotation vector `rvec` and translation vector `tvec`



When Does PnP Work Well?

1

Calibrated Camera

Intrinsic parameters (focal length, principal point) must be known accurately.

2

Accurate Keypoints

Detected 2D points must correspond precisely to their 3D counterparts on the model.

3

Minimal Occlusion

Enough visible keypoints must be available — heavy occlusion breaks the solver.

4

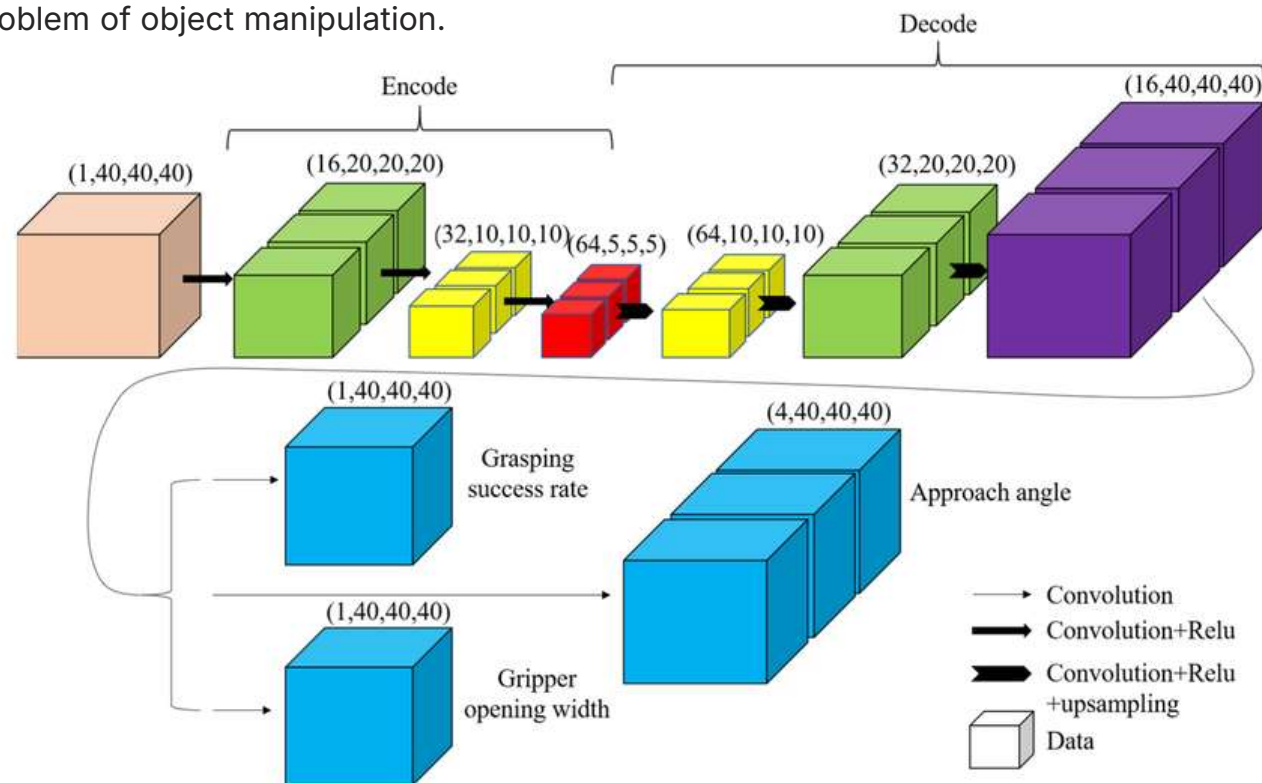
Reliable 3D Model

Object dimensions and keypoint positions must be trusted. CAD models or measured geometry work best.

PART 6

Deep Learning for 6D Pose

Bringing AI to the problem of object manipulation.



What Does the AI Model Predict?



Direct Regression

The network outputs (x, y, z, r, p, y) directly. Simple pipeline but requires the loss function to handle rotation carefully.



Keypoint Prediction

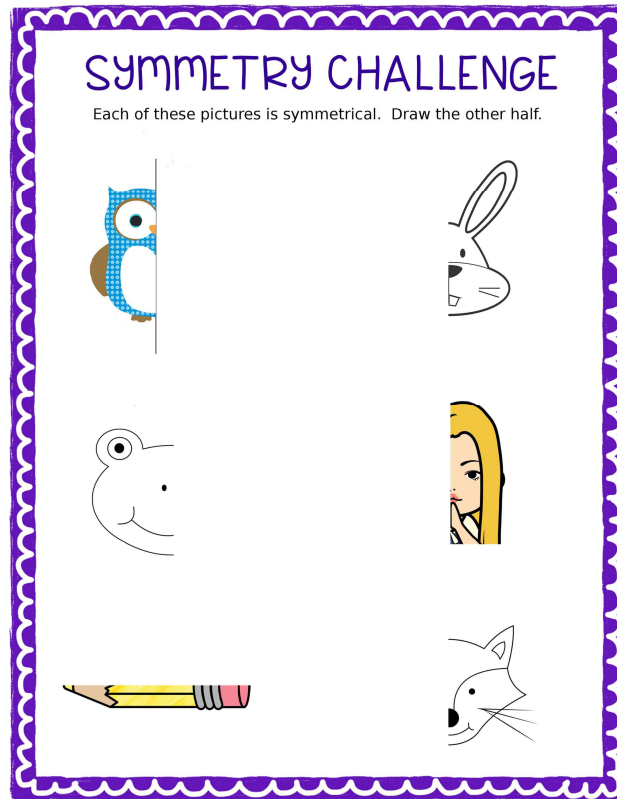
Network detects 2D keypoints; PnP solves the pose in a post-processing step. Clean separation of learning and geometry.



Dense Correspondences

Each pixel is mapped to a location on the 3D model surface. More robust but computationally expensive.

The Symmetry Challenge



Ambiguous Orientations

Symmetric objects — like a round bottle or a plain cylinder — can produce nearly identical images at multiple orientations. This creates ambiguity for the model.

Impact on training: Multiple valid ground-truth poses exist for a single image, making loss computation tricky.

Practical note: In robotics, symmetry is sometimes acceptable — if the gripper can grasp from any symmetric angle, the problem is less severe.

Occlusion and Clutter

Partial Occlusion

When part of the object is hidden, the model sees incomplete features. Accurate pose estimation becomes much harder without the full shape.

Cluttered Environments

Busy backgrounds or stacked objects confuse feature matching, leading to incorrect correspondences and degraded pose accuracy.

Training Data Must Match Reality

Models trained only on clean, isolated objects fail in real warehouses and factories. Training datasets must include occluded and cluttered scenes.

PART 7

Evaluating Pose Estimation

Knowing how accurate a prediction needs to be — and how to measure it.



Translation and Rotation Errors

Translation Error

Measures the Euclidean distance between the predicted and ground-truth position vectors. Expressed in meters or millimeters. A robot might tolerate ± 5 mm depending on the task.

Rotation Error

Measures the angular difference between predicted and true orientation. Even small errors in angle can cause the gripper to miss — both errors matter jointly for a successful grasp.

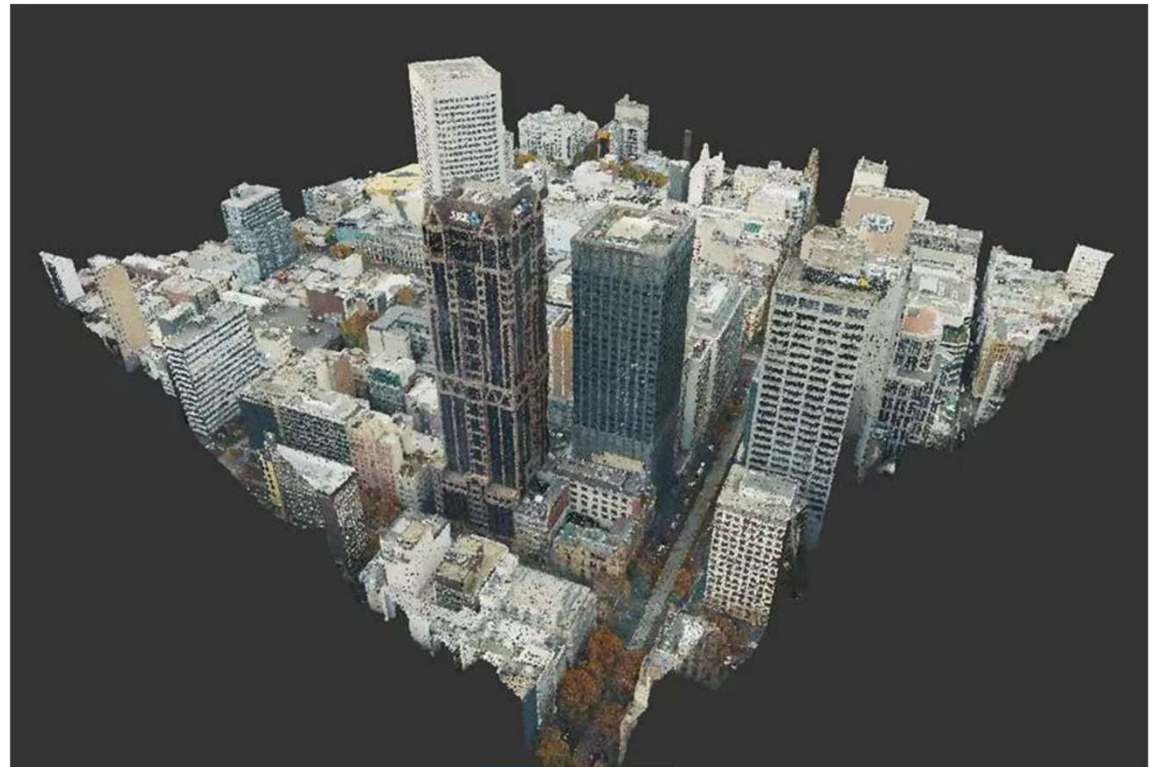
ADD Metric — The Intuition

Average Distance of Model Points

ADD transforms all 3D model points using both the ground-truth pose and the predicted pose, then measures the **mean distance** between corresponding transformed points.

If this average distance is **below a threshold** (e.g., 10% of the object's diameter), the prediction is counted as correct.

Students only need to understand the geometric meaning — no need to memorize the formula at this stage.



Good Enough for Robotics

→ Task-Driven Evaluation

Achieving the top benchmark score is not always the goal. The real question is: **is the pose accurate enough for the robot to succeed at its task?**

→ Application Over Publication

Task-driven evaluation is often more meaningful than chasing metrics in research papers. A system that grasps reliably is already valuable.

→ Success Criterion

If the robot consistently completes the grasp without failures, the pose estimation system has achieved its purpose — regardless of its exact benchmark ranking.

PART 8

Hands-On Python Labs

Getting familiar with pose representations and application pipelines through code.

Lab 1: Build a Pose Matrix

Construct a homogeneous 4×4 transformation matrix from scratch using NumPy:

```
import numpy as np

T = np.eye(4)          # Start with identity matrix

T[0, 3] = 0.25         # x translation: 25 cm
T[1, 3] = 0.10         # y translation: 10 cm
T[2, 3] = 0.40         # z translation: 40 cm

# T[:3, :3] holds the rotation matrix (identity = no rotation)
print(T)
```

This creates a pose with pure translation and zero rotation — the simplest possible 6D pose.

Lab 2: Read Pose from an AI Model

A typical AI model returns pose as a Python dictionary:

```
pose = {
    "x": 0.32,    # meters
    "y": -0.05,
    "z": 0.41,
    "roll": 0.10, # radians
    "pitch": -0.02,
    "yaw": 1.57   # ~90 degrees
}

print(f"Position: ({pose['x']}, {pose['y']}, {pose['z']})")
print(f"Orientation (RPY): ({pose['roll']}, {pose['pitch']}, {pose['yaw']})")
```

Parse this output and convert it to a rotation matrix or homogeneous transform for downstream use.

Lab 3: Compute Box Center from Detection

Bridge detection output to pose estimation by computing the 2D region of interest:

```
# Assume x1, y1 = top-left corner; x2, y2 = bottom-right corner
u = int((x1 + x2) / 2)
v = int((y1 + y2) / 2)

# Crop the ROI around the detected object
roi = image[y1:y2, x1:x2]

# Feed ROI into pose estimation model
# pose = pose_model.predict(roi)
```

Using the bounding box center as the ROI anchor is a simple and effective way to connect Week 4's detector to this week's pose estimator.

Lab 4: solvePnP with OpenCV

Apply the classical PnP solver to known 2D–3D correspondences:

```
ret, rvec, tvec = cv2.solvePnP(  
    object_points,    # Nx3 array of 3D model points  
    image_points,     # Nx2 array of 2D image points  
    camera_matrix,    # 3x3 intrinsic matrix K  
    dist_coeffs       # lens distortion coefficients  
)  
  
# Convert rotation vector to rotation matrix  
R, _ = cv2.Rodrigues(rvec)
```

`rvec` is a compact rotation vector (Rodrigues form); `tvec` is the translation. Together they define the full 6D pose relative to the camera.

Lab 5: Full Integration Pipeline Idea

```
# Step 1: Detect the object using a trained detector
bbox, class_id = detector.predict(image)

# Step 2: Estimate 6D pose relative to the camera frame
pose_in_camera = pose_estimator.predict(image, bbox)

# Step 3: Convert pose from camera frame to robot base frame
# Requires hand-eye calibration matrix T_robot_camera
pose_in_robot = T_robot_camera @ pose_in_camera

# Step 4: Send target pose to IK solver / motion planner
robot_arm.move_to(pose_in_robot)
```

This four-step pipeline is the conceptual backbone of any vision-guided robotic manipulation system. Each module maps directly to a course topic.

END OF WEEK 5

From Bounding Box to Grasp Pose

Week 5 is the pivotal step where the robot truly **understands** the spatial state of an object — not just its identity — before attempting to manipulate it.

📄 **Assignment:** Describe the full pipeline — detect → estimate pose → transform to robot frame → send to IK / motion planner — with a diagram and code sketch for each step.

